

Programación y Administración de Sistemas

Práctica 2. Expresiones regulares para programación de la shell.

Convocatoria de junio (curso 2023/2024)

Víctor Manuel Vargas Yun / M^a Isabel Jiménez Velasco

18 de marzo de 2024

Resumen

Esta serie de ejercicios se os entregan para que podáis practicar y profundicéis vuestros conocimientos de *bash* de cara al examen de prácticas. Estos ejercicios no se entregan, la evaluación de la práctica 2 se realizará mediante ejercicios similares a los expuestos en este guion. Para evitar problemas al ejecutar tus ejercicios de cara al examen, asegúrate de que todos los scripts que realices funcionen correctamente en los ordenadores de la UCO o conectándote por *ssh* al `ts.uco.es`. Para cualquier duda de los ejercicios, por favor, escribid en el foro del moodle o enviad un correo a la dirección `vvargas@uco.es` o `isajimenez@uco.es`

1. ejercicio1.sh

Crear un *script* que ejecute los comandos adecuados de *grep* que permitan realizar las siguientes tareas sobre el fichero de ejemplo *peliculas.txt*. El *script* recibirá el nombre del fichero por la línea de comandos. Hacer las comprobaciones de argumentos necesarias.

1. Mostrar el título de las películas que tengan una longitud de 4 palabras. (Los títulos siempre comienzan por "> ")
2. Mostrar las duraciones de películas que sean superiores a 1h 45min. La duración de las películas se muestra con uno o más dígitos al principio de la línea, un espacio y la secuencia "min".
3. Contar cuántas películas hay de cada país, suponiendo que el país siempre está encerrado entre guiones (por ejemplo, "-Estados Unidos-").
4. Mostrar aquellas palabras que contengan la letra "d", "l" o "t", una vocal y la misma letra (por ejemplo, "Universidad", "expectativas", "dedicarse", etc).
5. Mostrar todas aquellas líneas que terminen con 3 puntos ("...") y no empiecen por espacio, utilizando el operador de repetición {}.

A continuación, se muestra un ejemplo de la salida de este *script* :

```
1 i72jivem@VTS3:~/PAS/p2$ ./ejercicio1.sh
2 Argumentos erróneos . Uso: ./ejercicio1.sh <fichero_peliculas>.
3
4 i72jivem@VTS3:~/PAS/p2$ ./ejercicio1.sh peli.txt
5 Se esperaba un fichero del tipo peliculas.txt
6
7 i72jivem@VTS3:~/PAS/p2$ ./ejercicio1.sh peliculas.txt
8 1) Título de las películas que tengan una longitud de 4 palabras:
9 > Kong: La Isla Calavera
10 > Es por tu bien
```

```

11
12 2) Duraciones superiores a 1h 45min
13 1hr 59min
14 2hr 17min
15 2hr 09min
16 1hr 48min
17
18 3) Numero de peliculas por pais
19     2 -Espana-
20     3 -Estados Unidos-
21     1 -Rusia-
22
23 4) Lineas que contengan d, l o t , una vocal , y misma letra
24 soldados
25 ciudad
26 realidad
27 expectativas
28 escondidos
29 lila
30 oportunidad
31
32 5) Lineas que acaben con 3 puntos y no empiecen por espacios
33 Reparto: Tom Hiddleston, Samuel L. Jackson, Brie Larson...
34 Reparto: Hugh Jackman, Patrick Stewart, Dafne Keen...
35 Reparto: Jos Coronado, Javier C mara, Roberto lamo ...
36 Reparto: Marta Etura, Elvira Minguez, Nene...
37 Reparto: Matthew McConaughey, Reese Witherspoon, Seth MacFarlane...
38 Reparto: Documentary, Gabe Polsky...

```

2. ejercicio2.sh

Utilizando *sed*, hacer un script que, dado el fichero de texto *peliculas.txt* (recibido por línea de comandos), elimine las líneas vacías, los subrayados y lo formatee de la siguiente manera por cada película:

```

Título: XXX
| -> Fecha de estreno: XXX
| -> Director: XXX
| -> Reparto: XXX
| -> Duración: XXX

```

Además, el script deberá mostrar la salida por terminal y además guardarla en un *.txt* denominado *peliculas_formateadas.txt*, utilizando el comando *tee*.

A continuación, se muestra un ejemplo de la salida de este *script* :

```

1 i72jivem@VTS3:~/PAS/p2$ ./ejercicio2.sh
2 Argumentos erroneos . Uso: ./ejercicio2.sh <fichero_peliculas>.
3
4 i72jivem@VTS3:~/PAS/p2$ ./ejercicio2.sh pelis.txt
5 Se esperaba un fichero del tipo peliculas.txt
6
7 i72jivem@VTS3:~/PAS/p2$ ./ejercicio2.sh peliculas.txt
8 Titulo: Kong: La Isla Calavera
9 |->Fecha de estreno: (10/03/2017)
10 |->Director: Jordan Vogt-Roberts
11 |->Reparto: Tom Hiddleston, Samuel L. Jackson, Brie Larson...
12 |->Duracion: 1hr 59min
13 Titulo: Logan
14 |->Fecha de estreno: (03/03/2017)
15 |->Director: James Mangold
16 |->Reparto: Hugh Jackman, Patrick Stewart, Dafne Keen...
17 |->Duracion: 2hr 17min
18 Titulo: Es por tu bien
19 |->Fecha de estreno: (24/02/2017)
20 |->Director: Carlos Theron
21 |->Reparto: Jose Coronado, Javier Camara, Roberto Alamo...
22 |->Duracion: 1hr 33min
23 Titulo: El guardián Invisible
24 |->Fecha de estreno: (03/03/2017)
25 |->Director: Fernando Gonzalez Molina
26 |->Reparto: Marta Etura, Elvira Minguez, Nene...

```

```

27 |->Duracion: 2hr 09min
28 Titulo: Canta !
29 |->Fecha de estreno: (22/12/2016)
30 |->Director: Garth Jennings
31 |->Reparto: Matthew McConaughey, Reese Witherspoon, Seth MacFarlane...
32 |->Duracion: 1hr 48min
33 Titulo: Red Army
34 |->Fecha de estreno: (29/01/2015)
35 |->Director: Gabe Polsky
36 |->Reparto: Documentary, Gabe Polsky...
37 |->Duracion: 1hr 25min
38
39 Fichero peliculas_formateadas.txt creado con exito.

```

3. ejercicio3.sh

La eficiencia y rapidez con la que los datos viajan de un punto a otro es crucial para mantener una experiencia de usuario fluida y efectiva. Cada vez que accedemos a un sitio web o utilizamos un servicio en línea, nuestros datos inician un viaje a través de múltiples nodos en la red, conocidos como “saltos”, antes de alcanzar su destino final. Este proceso, aunque ocurre en milisegundos, es fundamental para todo, desde cargar una página web hasta transmitir un video en tiempo real. Para asegurar una navegación óptima y eficiente, es esencial que estos saltos sean lo más directos y rápidos posible. Utilizando **traceroute**, una herramienta de diagnóstico que traza la ruta que los paquetes de datos toman hasta un destino especificado, podemos obtener una visión detallada de cada salto en el camino.

Desarrolla un *script* de Bash que ejecute traceroute para una lista de direcciones IP proporcionadas por un archivo de .txt “ips.txt”. Este fichero se leerá por línea de comandos así como el tiempo máximo (segundos) de espera de respuesta en un salto. El script deberá calcular el tiempo medio de respuesta en el primer salto de cada destino, y luego mostrará los destinos ordenados de manera descendente basándose en este tiempo medio. Para los casos en los que en el primer salto no se obtiene respuesta en el tiempo establecido, se mostrará un mensaje de error.

Este script permitirá a los usuarios identificar cuál de los destinos proporcionados tiene el mejor tiempo de respuesta en el primer salto, lo cual puede ser un indicador de proximidad o de una conexión de red eficiente.

Hacer las comprobaciones de argumentos que considere oportunas.

Nota: las operaciones aritméticas en bash no admiten operaciones con números decimales. Para realizar este tipo de operaciones, puedes usar la herramienta *bc*, que resuelve una operación que esté indicada como cadena de texto. Por ejemplo: `echo "5.3 + 2.4" | bc` dará como resultado 7.7.

A continuación se muestra un ejemplo del tipo de salida esperada:

```

1 i72jivem@VTS3:~/PAS/p2$ ./ejercicio3.sh
2 Argumentos erroneos. Uso: ./ejercicio3.sh <archivo_con_IPs><timeout>
3
4 i72jivem@VTS3:~/PAS/p2$ ./ejercicio3.sh ip.txt 1
5 Se esperaba un fichero del tipo ips.txt
6
7 i72jivem@VTS3:~/PAS/p2$ ./ejercicio3.sh ips.txt 1
8 IP 192.168.100.100 ha tardado 37.78 ms en el primer salto
9 IP 149.112.112.112 ha tardado 12.53 ms en el primer salto
10 IP 208.67.222.222 ha tardado 11.31 ms en el primer salto
11 IP 185.228.168.9 ha tardado 11.13 ms en el primer salto
12 IP 192.168.1.1 ha tardado 11.09 ms en el primer salto
13 IP 8.8.8.8 ha tardado 10.85 ms en el primer salto
14 IP 10.0.0.1 ha tardado 10.58 ms en el primer salto
15 IP 123.123.123.123 ha tardado 10.56 ms en el primer salto
16
17 Error: No se recibí respuesta para 234.234.234.234 en 1 segundos

```

4. ejercicio4.sh

Escribir un *script* que realice lo siguiente:

1. Listar todos los ficheros ocultos de la carpeta personal del usuario ordenados de menor a mayor número de caracteres
2. Listar por pantalla todos los procesos que el usuario del sistema está ejecutando en ese momento. Para cada proceso tendrá que mostrar el PID, hora en que se lanzó y nombre del fichero ejecutable. Ordénalos de forma alfabética inversa por nombre de ejecutable. (Deberás consultar información de ps)

A continuación se muestra un ejemplo del tipo de salida esperada:

```
1 i72jivem@VTS3:~/PAS/p2$ ./ejercicio4.sh
2 1) Ficheros ocultos de /home/i72jivem
3 .
4 ..
5 .m2
6 .qt
7 .ddd
8 .kde
9 .pki
10 .ssh
11 .atom
12 .dbus
13 .dmrc
14 .gvfs
15 .java
16 .mcop
17 .adobe
18 .cache
19 .cshrc
20 .gconf
21 .hplip
22 .icons
23 .local
24 .login
25 .nedit
26 .pulse
27 .config
28 .gnome2
29 .imagej
30 .mcoprc
31 .pencil
32 .themes
33 .tomcat
34 .vscode
35 .android
36 .eclipse
37 .lessht
38 .mozilla
39 .nemiver
40 .phpbrew
41 .esd_auth
42 .gimp-2.8
43 .nautilus
44 .netbeans
45 .evolution
46 .filezilla
47 .firestorm
48 .gitconfig
49 .gitkraken
50 .install4j
51 .wget-hsts
52 .ddcccontrol
53 .subversion
54 .thumbnails
55 .Xauthority
56 .bash_logout
57 .gnuplot-wxt
58 .octave_hist
59 .python-eggs
60 .bash_history
61 .ICEauthority
62 .kompozer.net
```

```

63 .packettracer
64 .pulse-cookie
65 .vscode-server
66 .xmlcopyeditor
67 .gnome2_private
68 .gstreamer-0.10
69 .gnuplot_history
70 .mission-control
71 .rstudio-desktop
72 .AndroidStudio3.0
73 .oracle_jre_usage
74
75 ----
76
77 2) Listado de los procesos ejecutados por el usuario i72jivem
78 PID: "101990" Hora: "12:23" Ejecutable: "/usr/local/sbin/sftp-server"
79 PID: "59003" Hora: "14:54" Ejecutable: "sshd: i72jivem@pts/0"
80 PID: "101989" Hora: "12:23" Ejecutable: "sshd: i72jivem@notty"
81 PID: "103226" Hora: "15:36" Ejecutable: "ps ux"
82 PID: "103006" Hora: "15:36" Ejecutable: "/bin/bash ./ejercicio4.sh"
83 PID: "59004" Hora: "14:54" Ejecutable: "-bash"

```

5. ejercicio5.sh

Desarrollar un *script* que reciba como argumento el nombre de un fichero de texto y extraiga las palabras que se encuentren en las líneas que contienen números. Cada palabra se deberá mostrar en una línea diferente y deberán estar ordenadas por orden alfabético inverso. Además de la palabra, en cada línea se mostrará el número de orden y el número de veces que se repite la palabra (sin distinguir mayúsculas). Recuerda realizar los controles de errores oportunos.

Nota 1: para quedarte con los elementos únicos de una “lista ordenada” puedes usar el comando *uniq* mediante una tubería.

Nota 2: Para mostrar el número de orden de cada palabra puedes usar el comando *nl* mediante una tubería.

A continuación se muestra un ejemplo de salida:

```

1 i72jivem@VTS3:~/PAS/p2$ ./ejercicio5.sh
2 Argumentos incorrectos. Uso: ./ejercicio5.sh <texto.txt>
3
4 i72jivem@VTS3:~/PAS/p2$ ./ejercicio5.sh tt.txt
5 Se esperaba un fichero del tipo texto.txt
6
7 i72jivem@VTS3:~/PAS/p2$ ./ejercicio5.sh texto.txt
8 #          Count          Word
9      1              5 y
10     2              1 veces
11     3              1 un
12     4              2 tr
13     5              2 texto
14     6              1 tail
15     7              1 su
16     8              1 sistemas
17     9              1 signos
18    10              2 sed
19    11              1 proposito
20    12              1 programacion
21    13              1 practicas
22    14              2 practica
23    15              1 posibilidades
24    16              1 pense
25    17              3 para
26    18              1 palabras
27    19              2 numeros
28    20              1 numericos
29    21              1 manipular
30    22              1 manera
31    23              1 manejo
32    24              1 las
33    25              3 la
34    26              1 incluye
35    27              1 incluso
36    28              2 grep
37    29              1 formatos

```

38	30	1 facilitar
39	31	2 este
40	32	1 especificamente
41	33	1 esenciales
42	34	2 es
43	35	2 en
44	36	1 el
45	37	1 ejemplos
46	38	1 eficiente
47	39	1 dise ado
48	40	1 diferentes
49	41	1 del
50	42	7 de
51	43	1 contiene
52	44	1 conceptos
53	45	2 con
54	46	1 comprension
55	47	1 complejidad
56	48	2 como
57	49	1 clase
58	50	2 bash
59	51	1 archivo
60	52	1 a o
61	53	1 a adir
62	54	1 algunos
63	55	1 administracion
64	56	1 ademas

6. ejercicio6.sh

Desarrollar un *script* que muestre el contenido del fichero */etc/passwd/* (y parte del fichero */etc/group/*) de forma amigable. El script recibirá un único argumento que será una cadena de texto que corresponde con un tipo de shell. Sólo se mostrarán los usuarios que tengan esa cadena como shell del sistema de la siguiente manera:

- Logname del usuario
- UID del usuario
- Groupname de su grupo primario (aquí tendrás que acceder a */etc/group/* sabiendo su GID)
- GID del grupo primario
- Shell por defecto

A continuación se muestra un ejemplo de salida:

```

1 i72jivem@VTS3:~/PAS/p2$ ./ejercicio6.sh
2 Argumentos incorrectos. Uso: ./ejercicio6.sh </bin/bash>
3
4 i72jivem@VTS3:~/PAS/p2$ ./ejercicio6.sh /bin/bash
5 =====
6 Logname: root
7 ->UID: 0
8 ->Grupo: root
9 ->GID: 0
10 ->Shell por defecto: /bin/bash
11
12 =====
13 Logname: couchdb
14 ->UID: 106
15 ->Grupo: couchdb
16 ->GID: 113
17 ->Shell por defecto: /bin/bash
18
19
20
21 i72jivem@VTS3:~/PAS/p2$ ./ejercicio6.sh /bin/sh
22 =====
23 Logname: daemon
24 ->UID: 1
25 ->Grupo: daemon

```

```

26 |->GID: 1
27 |->Shell por defecto: /bin/sh
28
29 =====
30 Logname: bin
31 |->UID: 2
32 |->Grupo: bin
33 |->GID: 2
34 |->Shell por defecto: /bin/sh
35
36 =====
37 Logname: sys
38 |->UID: 3
39 |->Grupo: sys
40 |->GID: 3
41 |->Shell por defecto: /bin/sh
42
43 =====
44 Logname: games
45 |->UID: 5
46 |->Grupo: games
47 |->GID: 60
48 |->Shell por defecto: /bin/sh
49
50 =====
51 Logname: man
52 |->UID: 6
53 |->Grupo: man
54 |->GID: 12
55 |->Shell por defecto: /bin/sh
56
57 =====
58 Logname: lp
59 |->UID: 7
60 |->Grupo: lp
61 |->GID: 7
62 |->Shell por defecto: /bin/sh
63
64 ...
65
66 =====
67
68 Logname: irc
69 |->UID: 39
70 |->Grupo: irc
71 |->GID: 39
72 |->Shell por defecto: /bin/sh
73
74 =====
75 Logname: gnats
76 |->UID: 41
77 |->Grupo: gnats
78 |->GID: 41
79 |->Shell por defecto: /bin/sh
80
81 =====
82 Logname: nobody
83 |->UID: 65534
84 |->Grupo: nogroup
85 |->GID: 65534
86 |->Shell por defecto: /bin/sh
87
88 =====
89 Logname: libuuid
90 |->UID: 100
91 |->Grupo: libuuid
92 |->GID: 101
93 |->Shell por defecto: /bin/sh
94
95 =====
96 Logname: speech-dispatcher
97 |->UID: 108
98 |->Grupo: audio
99 |->GID: 29
100 |->Shell por defecto: /bin/sh

```